

Smart HAS

Monitoramento de Hipertensão Arterial Sistêmica

Fase 6 — Oracle PL/SQL e integração com o back-end Java

Documentação técnica da entrega

Integrante do grupo

Gabriel Garuti Paiva Cracco — RM 554866

Repositório: <https://github.com/GabrielGaruti/smarthas>

Vídeo: [inserir aqui o link do vídeo da Fase 6 no YouTube]

1. Visão geral da Fase 6

O Smart HAS é uma solução para monitoramento da Hipertensão Arterial Sistêmica: o paciente registra leituras de pressão (manualmente ou por um monitor conectado), o sistema classifica cada leitura e apoia a decisão com recomendações, alertas e a unidade de saúde mais próxima.

Até a Fase 5 o projeto tinha um MVP funcional e conectado: app mobile em React Native, API REST em Java/Spring Boot e painel administrativo em Angular. Nesta fase o foco foi levar parte da inteligência para o banco de dados, adotando o **Oracle** como camada de persistência e implementando **procedures e functions PL/SQL** reutilizáveis, chamadas diretamente pelo back-end Java.

1.1 Entregas desta fase

Parte	Entrega	Onde está no repositório
1 — Aprimoramento	Módulo de alertas clínicos, relatórios, indicadores, leituras de sensor IoT, refatorações e padrões arquiteturais	<code>backend-springboot, web-angular, mobile-react-native</code>
2 — Banco Oracle	Modelo lógico/físico com 7 tabelas, DER, carga de dados simulados	<code>database/oracle/01_ddl_tabelas.sql, 04_carga_dados.sql, database/der</code>
3 — PL/SQL	4 functions (2 exigidas + apoio) e 3 procedures de negócio (+ procedure de log), integradas ao Java via JDBC	<code>database/oracle/02_functions.sql, 03_procedures.sql, integration/</code>

1.2 Arquitetura

A API continua sendo o ponto central consumido pelos dois clientes. A novidade é a porta de integração **ClinicalRulesGateway**: a aplicação conversa apenas com essa interface, e o Spring injeta o adaptador conforme o perfil ativo. No perfil **oracle**, o adaptador **OraclePlsqlGateway** executa as rotinas PL/SQL via JDBC; no perfil padrão (H2), o **JavaRulesGateway** reproduz as mesmas regras em Java, permitindo desenvolver e testar sem acesso ao servidor Oracle.

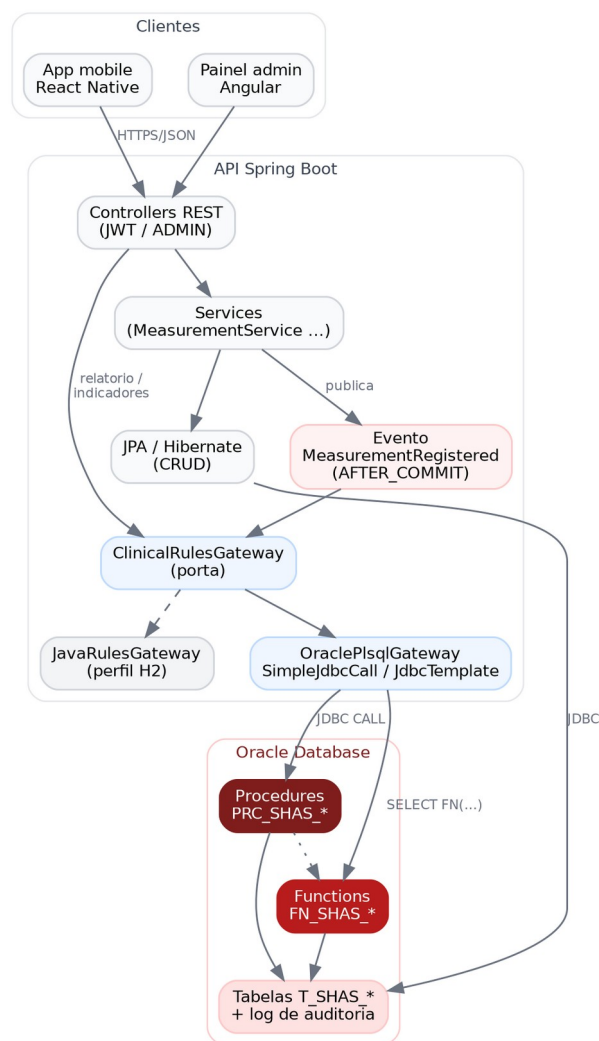


Figura 1 — Arquitetura da solução e fluxo REST → Java → JDBC → Oracle

2. Parte 1 — Aprimoramento da solução

2.1 Novas funcionalidades

Funcionalidade	Descrição	Valor agregado
Alertas clínicos automáticos	Toda leitura registrada é avaliada no banco. Leituras $\geq 180 \times 120$ geram alerta CRÍTICO (crise hipertensiva); $\geq 140 \times 90$ geram MODERADO, ou ALTO quando 3 das 5 últimas leituras indicam hipertensão. O alerta volta na própria resposta do POST e aparece no app.	O paciente é avisado no momento da medição; a equipe enxerga quem precisa de atenção primeiro.
Gestão de alertas	Endpoints <code>GET /alerts</code> e <code>PATCH /alerts/{id}/resolve</code> ; aba "Alertas" no app e tela "Alertas & Relatórios" no painel, com filtro por status.	Fluxo completo: detectar, acompanhar e encerrar.
Relatório resumido por paciente	O administrador gera, pelo painel, um relatório consolidado (média, máxima, % na meta, alertas abertos e nível de risco) executado pela procedure no Oracle.	Rotina batch no banco, rápida e auditável, sem trafegar todas as leituras para a aplicação.
Indicador "leituras na meta"	Percentual de leituras abaixo de 140×90 nos últimos 30 dias, calculado por function e exibido no app e no dashboard.	Indicador clínico simples de adesão ao tratamento.
Leituras de sensor IoT	Nova entidade de dispositivo, origem da leitura (MANUAL/SENSOR) e frequência cardíaca opcional; procedure que importa leituras simuladas.	Prepara o sistema para monitores de pressão conectados.

2.2 Refatorações e padrões aplicados

- **Ports & Adapters + Strategy por perfil:** `ClinicalRulesGateway` isola o motor de regras. Trocar Oracle por Java (ou por outro banco) não altera controllers nem services.
- **Evento de domínio:** `MeasurementService` publica `MeasurementRegisteredEvent`; o `MeasurementAlertListener` reage com `@TransactionalEventListener(AFTER_COMMIT)` e transação própria (`REQUIRES_NEW`). A procedure só roda depois que a leitura está gravada, e uma falha na regra não desfaz a medição do paciente.
- **Modelo temporal:** data e hora, antes duas Strings, viraram um único `LocalDateTime` (`DT_MEDICAO_TIMESTAMP`). Isso permite ordenar e filtrar por período no banco. O contrato JSON (`date/time`) foi mantido para não quebrar os clientes.
- **Mapeamento JPA alinhado ao Oracle:** entidades com `@Table/@Column` no padrão FIAP (`T_SHAS_, ID_, VL_, DT_...`) e conversor `boolean` $\leftrightarrow$ `NUMBER(1)`.
- **Governança do schema:** no perfil `oracle` o Hibernate não altera tabelas (`ddl-auto=none`); o DDL é versionado em scripts SQL.
- **Transações explícitas** (`@Transactional`, inclusive `readOnly`) e **idempotência** dos alertas (constraint única por leitura e tipo).
- **Tratamento de erros do banco:** `RAISE_APPLICATION_ERROR` (ORA-20000 a 20999) vira HTTP 400 com a mensagem da regra, sem expor o stack do Oracle.
- **Credenciais fora do código:** usuário/senha do Oracle lidos de variáveis de ambiente (`ORACLE_USER`, `ORACLE_PASSWORD`, `ORACLE_URL`).

- **Validação reforçada:** formato de data/hora e faixa da frequência cardíaca com Bean Validation.

2.3 Mudanças por camada

Camada	Principais arquivos novos/alterados
Back-end	<code>integration/</code> (gateway, adaptadores Oracle e Java), <code>event/</code> (evento e listener), <code>Alert</code> , <code>SummaryReport</code> , <code>AlertService</code> , <code>ReportService</code> , <code>AlertController</code> , <code>AdminController</code> , <code>application-oracle.properties</code> , <code>pom.xml</code> (driver <code>ojdbc11</code>)
Painel Angular	Nova tela <code>clinical.component.ts</code> (indicadores, relatório e alertas); dashboard com indicador PL/SQL; medições exibem origem e alerta gerado
App mobile	Nova aba <code>AlertsScreen</code> ; Home com "% na meta" e resumo do paciente; formulário com frequência cardíaca e aviso de alerta ao salvar
Banco	Pasta <code>database/</code> com scripts 00 a 05, DER e README

3. Parte 2 — Integração do banco Oracle

3.1 Modelo relacional (DER)

O modelo representa as entidades principais do Smart HAS. O usuário (paciente ou administrador) registra medições, que podem vir de um dispositivo IoT; cada medição pode gerar alertas; e os relatórios consolidam os indicadores por paciente. Unidades de saúde e o log de processamento são tabelas independentes.

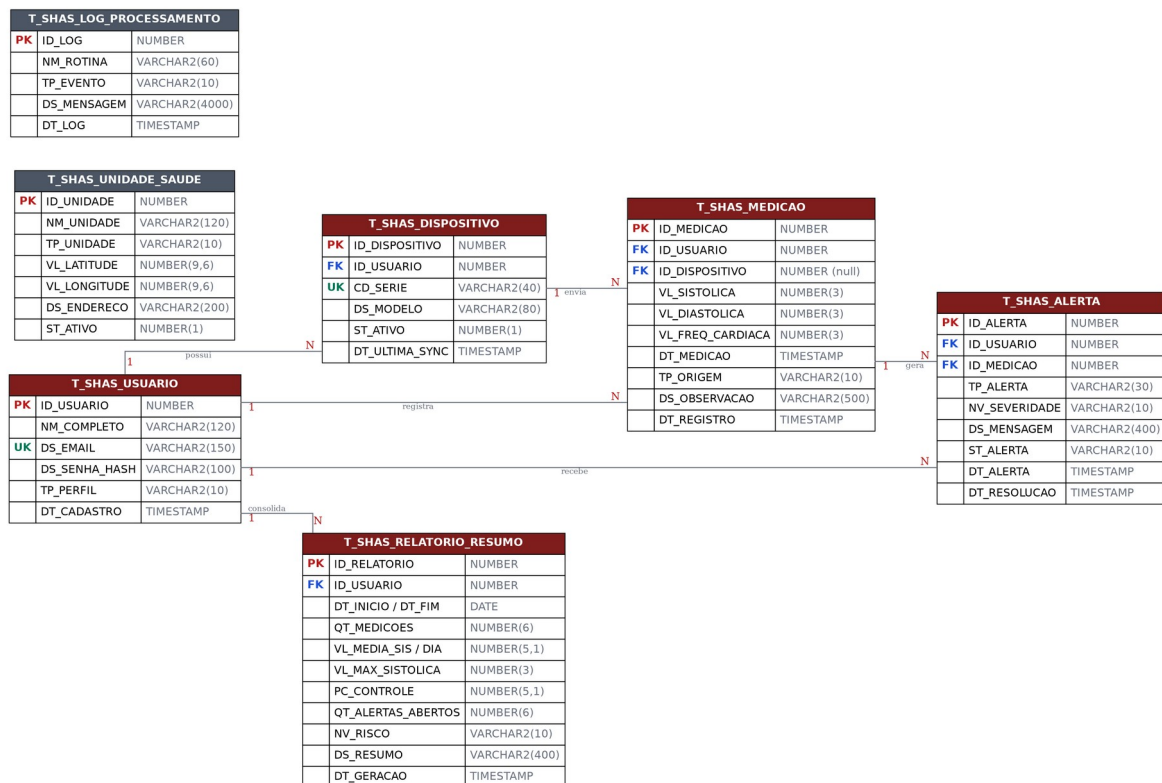


Figura 2 — DER do Smart HAS (fonte em database/der/smarthas_der.dot)

3.2 Tabelas criadas

Tabela	Finalidade	Regras e constraints principais
T_SHAS_USUARIO	Pacientes e administradores	PK identity; e-mail único; perfil USER/ADMIN; senha em BCrypt
T_SHAS_UNIDADE_SAUDE	Hospitais, UBS e sensores públicos do mapa	Tipo HOSPITAL/CLINIC/SENSOR; coordenadas NUMBER(9,6); ST_ATIVO 0/1
T_SHAS_DISPOSITIVO	Monitores de pressão IoT do paciente	FK usuário; número de série único; ST_ATIVO 0/1
T_SHAS_MEDICAO	Leituras de pressão	FK usuário e dispositivo (opcional); sistólica 50–300; diastólica 30–200; FC 30–220; origem MANUAL/SENSOR; índice (usuário, data)
T_SHAS_ALERTA	Alertas gerados pela procedure	FK medição com ON DELETE CASCADE; única por (medição, tipo); severidade e status validados por CHECK

Tabela	Finalidade	Regras e constraints principais
T_SHAS_RELATORIO_RESUMO	Relatórios consolidados	FK usuário; risco SEM DADOS/BAIXO/MODERADO/ALTO
T_SHAS_LOG_PROCESSAMENTO	Auditoria das rotinas PL/SQL	Evento INFO/AVISO/ERRO; mensagem até 4000 caracteres

Todas as chaves usam **GENERATED BY DEFAULT ON NULL AS IDENTITY**, compatível com **GenerationType.IDENTITY** do JPA. As constraints seguem a nomenclatura **PK_**, **FK_**, **UK_**, **CK_** e **IX_**, e cada tabela tem **COMMENT** descritivo.

3.3 Carga de dados simulados

O script **04_carga_dados.sql** popula o banco com dados coerentes com o domínio. As datas são relativas a **SYSDATE**, então os indicadores dos últimos 30 dias sempre têm dados na demonstração.

Dado	Quantidade	Observação
Usuários	6	1 administrador e 5 pacientes; um paciente sem leituras para demonstrar o caso "SEM DADOS"
Unidades de saúde	5	Inclui uma unidade inativa (não é sugerida pela recomendação)
Dispositivos IoT	4	3 ativos e 1 inativo (usado no teste de exceção)
Leituras manuais	13	Histórico com casos normais, elevados, hipertensão e uma crise hipertensiva
Leituras de sensor	24	8 por dispositivo ativo, importadas pela procedure de simulação
Alertas	gerados	Criados pela própria procedure ao avaliar o histórico e as leituras importadas

3.4 Ordem de execução dos scripts

1. **01_ddl_tabelas.sql** — tabelas, constraints e índices.
2. **02_functions.sql** — procedure de log e functions.
3. **03_procedures.sql** — procedures de negócio.
4. **04_carga_dados.sql** — carga simulada e geração dos alertas.
5. **05_testes_e_consultas.sql** — uso das functions em SQL, execução das procedures e cenários de erro.

O script **00_drop.sql** remove todos os objetos para reexecução do zero.

4. Parte 3 — Procedures e functions PL/SQL

As rotinas seguem as mesmas boas práticas: parâmetros **IN/OUT** tipados, **RETURN** explícito nas functions, cabeçalho com objetivo e regras, uso de **%TYPE/%ROWTYPE**, códigos de erro próprios com **RAISE_APPLICATION_ERROR** e registro de falhas em log por meio de transação autônoma.

4.1 Functions

FN_SHAS_TAXA_CONTROLE — indicador

Objetivo: calcular o percentual de leituras do paciente dentro da meta terapêutica (sistólica < 140 e diastólica < 90) nos últimos N dias. **Parâmetros:** **p_id_usuario** IN NUMBER, **p_dias** IN NUMBER **DEFAULT** 30. **Retorno:** NUMBER de 0 a 100 com uma casa decimal, ou **NULL** quando não há leituras no período. **Exceções:** -20002 para período inválido e -20001 para usuário inexistente (a partir de **NO_DATA_FOUND**).

FN_SHAS_RESUMO_PACIENTE — dados formatados

Objetivo: devolver um texto-resumo pronto para exibição no app e no painel, reaproveitando as outras functions. **Exemplo de retorno:** "Paciente Demonstracao | 8 leituras em 30 dias | media 141/90 mmHg | 37,5% na meta | ultima: 01/10/2026 07:10 (Hipertensao)". **Exceções:** erros de negócio sobem intactos; erros inesperados são registrados em **T_SHAS_LOG_PROCESSAMENTO** e a function devolve uma mensagem amigável.

Functions de apoio

- **FN_SHAS_FORMATAR_MEDICAO(p_id_medicao):** linha legível de uma leitura (data, pressão, FC, classificação e origem).
- **FN_SHAS_CLASSIFICAR(p_sistolica, p_diastolica, p_formato):** mesma regra de classificação do enum Java, garantindo consistência entre as camadas. Declarada **DETERMINISTIC**.

Uso prático das functions em consultas SQL

```
-- Painel de pacientes: indicador e resumo direto no SELECT
SELECT u.NM_COMPLETO,
       FN_SHAS_TAXA_CONTROLE(u.ID_USUARIO, 30) AS PC_NA_META_30D,
       FN_SHAS_RESUMO_PACIENTE(u.ID_USUARIO, 30) AS RESUMO
FROM T_SHAS_USUARIO u
WHERE u.TP_PERFIL = 'USER'
ORDER BY PC_NA_META_30D NULLS LAST;

-- Function no WHERE: pacientes com controle abaixo de 60%
SELECT NM_COMPLETO FROM T_SHAS_USUARIO
WHERE TP_PERFIL = 'USER' AND FN_SHAS_TAXA_CONTROLE(ID_USUARIO, 30) < 60;

-- Function no GROUP BY: distribuicao das leituras por classificacao
SELECT FN_SHAS_CLASSIFICAR(VL_SISTOLICA, VL_DIASTOLICA, 'ROTULO') AS CLASSIFICACAO,
COUNT(*)
FROM T_SHAS_MEDICAO
GROUP BY FN_SHAS_CLASSIFICAR(VL_SISTOLICA, VL_DIASTOLICA, 'ROTULO');
```

A primeira consulta é exatamente a que o back-end executa no endpoint **GET /admin/patients/indicators**, que alimenta a tela de indicadores do painel Angular.

4.2 Procedures

PRC_SHAS_REGISTRAR_ALERTA — acionada pelo back-end Java

Objetivo: avaliar uma leitura recém-gravada e registrar um alerta clínico quando ela for crítica.

Parâmetros: `p_id_medicao` IN, `p_id_alerta` OUT, `p_severidade` OUT.

Condição (IF / ELSIF)	Tipo de alerta	Severidade
Sistólica ≥ 180 ou diastólica ≥ 120	CRISE_HIPERTENSIVA	CRÍTICO
$\geq 140 \times 90$ e 3+ das 5 últimas leituras com hipertensão (CURSOR + LOOP)	TENDENCIA_ALTA	ALTO
$\geq 140 \times 90$ (demais casos)	HIPERTENSAO	MODERADO
Abaixo de 140×90	—	NENHUM (não grava)

A tendência é verificada com um **cursor explícito** parametrizado (`OPEN / FETCH / EXIT WHEN %NOTFOUND / CLOSE`) que percorre as cinco leituras mais recentes. A procedure é **idempotente**: se a mesma leitura for reprocessada, o `DUP_VAL_ON_INDEX` é tratado e o alerta existente é devolvido. Ela não executa `COMMIT`, deixando o controle da transação para quem chama, e trata `NO_DATA_FOUND` (-20010) e `OTHERS` (fecha o cursor, registra no log e propaga o erro).

PRC_SHAS_GERAR_RELATORIO_RESUMO — acionada pelo painel administrativo

Objetivo: rotina batch que consolida os indicadores dos últimos N dias para todos os pacientes, ou para um paciente específico, e grava em `T_SHAS_RELATORIO_RESUMO`. **Parâmetros:** `p_dias` IN DEFAULT 30, `p_id_usuario` IN DEFAULT NULL, `p_qt_gerados` OUT.

- Percorre os pacientes com **cursor FOR LOOP** e, para cada um, calcula quantidade, médias, máxima e alertas abertos.
- Usa `FN_SHAS_TAXA_CONTROLE` e `FN_SHAS_RESUMO_PACIENTE`, reaproveitando a lógica das functions.
- Classifica o risco: SEM DADOS, ALTO (alerta crítico aberto ou taxa < 50%), MODERADO (taxa < 80%) ou BAIXO.
- Tem **tratamento de erro por paciente**: uma falha é registrada no log e não interrompe os demais. Ao final faz `COMMIT` e grava um resumo da execução no log.

PRC_SHAS_SIMULAR_LEITURAS_SENSOR — importação de dados IoT

Gera leituras plausíveis com `DBMS_RANDOM` para um dispositivo, distribuídas no período informado, e submete cada uma à `PRC_SHAS_REGISTRAR_ALERTA` (reuso da regra). Valida quantidade (-20013) e dispositivo inexistente ou inativo (-20012). Foi usada na carga do item 3.3.

PRC_SHAS_LOG — apoio

Grava eventos em `T_SHAS_LOG_PROCESSAMENTO` com `PRAGMA AUTONOMOUS_TRANSACTION`, de modo que o log persiste mesmo quando a transação principal sofre `ROLLBACK`.

4.3 Integração com o Java (REST → Java → JDBC → Oracle)

Fluxo do evento de back-end: quando o paciente salva uma medição pelo app, o `POST /measurements` chega ao `MeasurementService`, que grava a leitura via JPA e publica o evento `MeasurementRegisteredEvent`. Após o commit, o listener chama o gateway, que executa a

procedure no Oracle por JDBC. Antes de responder, o controller busca o alerta gerado e o devolve no JSON; o app então mostra o aviso ao paciente.

```
// MeasurementAlertListener.java - evento de back-end
@TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
@Transactional(propagation = Propagation.REQUIRES_NEW)
public void onMeasurementRegistered(MeasurementRegisteredEvent event) {
    AlertOutcome outcome = gateway.registerAlert(event.measurementId());
}

// OraclePlsqlGateway.java - chamada da procedure via SimpleJdbcCall
registerAlertCall = new SimpleJdbcCall(jdbc)
    .withProcedureName("PRC_SHAS_REGISTRAR_ALERTA")
    .withoutProcedureColumnMetaDataAccess()
    .declareParameters(
        new SqlParameter("P_ID_MEDICA0", Types.NUMERIC),
        new SqlOutParameter("P_ID_ALERTA", Types.NUMERIC),
        new SqlOutParameter("P_SEVERIDADE", Types.VARCHAR));

Map<String, Object> out = registerAlertCall.execute(
    new MapSqlParameterSource().addValue("P_ID_MEDICA0", measurementId));

// Functions usadas em SQL pelo JdbcTemplate
jdbc.queryForObject("SELECT FN_SHAS_TAXA_CONTROLE(?, ?) FROM DUAL",
    BigDecimal.class, userId, days);
```

Endpoint	Perfil	Rotina Oracle
POST /measurements	USER	PRC_SHAS_REGISTRAR_ALERTA (via evento)
GET /recommendations	USER	FN_SHAS_TAXA_CONTROLE, FN_SHAS_RESUMO_PACIENTE
POST /admin/reports/summary?days=30	ADMIN	PRC_SHAS_GERAR_RELATORIO_RESUMO
GET /admin/patients/indicators	ADMIN	SELECT com as duas functions
GET /alerts · PATCH /alerts/{id}/resolve	USER/ADMIN	Lê/atualiza T_SHAS_ALERTA

4.4 Códigos de erro e tratamento de exceções

Código	Origem	Significado
-20001	Functions	Usuário não encontrado
-20002	Functions	Período inválido
-20003	FN_SHAS_CLASSIFICAR	Sistólica/diastólica nulas
-20004	FN_SHAS_FORMATAR_MEDICAO	Medição não encontrada
-20010	PRC_SHAS_REGISTRAR_ALERTA	Medição não encontrada
-20011	PRC_SHAS_GERAR_RELATORIO_RESUMO	Período fora de 1 a 365 dias
-20012	PRC_SHAS_SIMULAR_LEITURAS_SENSOR	Dispositivo inexistente ou inativo
-20013	PRC_SHAS_SIMULAR_LEITURAS_SENSOR	Quantidade fora de 1 a 500

No Java, o `GlobalExceptionHandler` identifica esses códigos na `SQLException` e devolve HTTP 400 com a mensagem da regra; os demais erros de banco viram HTTP 500 genérico. O script `05_testes_e_consultas.sql` exercita três desses cenários.

5. Como executar

1. **Banco:** no SQL Developer, conectar com o usuário Oracle da FIAP e executar os scripts 01 a 05 de `database/oracle` (F5).
2. **API com Oracle:** definir `ORACLE_USER` e `ORACLE_PASSWORD` e rodar `mvn spring-boot:run -Dspring-boot.run.profiles=oracle` em `backend-springboot`. O Swagger fica em `http://localhost:8080/swagger-ui.html`.
3. **Painel Angular:** `npm install` e `npm start` em `web-angular`; acessar `http://localhost:4200` com `admin@smarthas.com / admin123` e abrir "Alertas & Relatórios".
4. **App mobile:** `npm install` e `npx expo start` em `mobile-react-native`, ajustando `API_URL` em `src/api/client.js`. Entrar com `paciente@smarthas.com / 123456`.

Sem acesso ao Oracle, basta omitir o perfil (`mvn spring-boot:run`): a API sobe com H2 e o `JavaRulesGateway`, e o dashboard indica o motor em uso ("Oracle PL/SQL" ou "Java (modo H2)").

5.1 Roteiro de demonstração

- Executar `05_testes_e_consultas.sql` e mostrar o resultado das functions e o log.
- No app, registrar 186x121 e mostrar o alerta CRÍTICO retornado na hora.
- No painel, abrir "Alertas & Relatórios", gerar o relatório e mostrar o motor `ORACLE_PLSQL` e o tempo de execução.
- Conferir no SQL Developer os registros novos em `T_SHAS_ALERTA` e `T_SHAS_RELATORIO_RESUMO`.

6. Conclusão

A Fase 6 consolidou a interoperabilidade entre mobile, web, API REST e banco Oracle. Regras que dependem de muitos dados, como o relatório consolidado e a tendência de hipertensão, passaram a rodar perto dos dados, em PL/SQL, enquanto a aplicação Java orquestra o fluxo, protege o acesso e expõe os resultados aos clientes. A porta de integração com dois adaptadores manteve o código desacoplado e testável, e os scripts versionados tornam o banco reproduzível.

Como próximos passos, o projeto pode agendar o relatório com `DBMS_SCHEDULER`, agrupar as rotinas em um `PACKAGE`, receber leituras reais dos monitores por Bluetooth e notificar o paciente por push quando um alerta crítico for gerado.

Links da entrega

GitHub: <https://github.com/GabrielGaruti/smarthas>

Vídeo (YouTube, não listado): **[inserir aqui o link do vídeo da Fase 6 no YouTube]**